

Hashing and Digital signatures

Class Test (Practical skill assessment)

Access and Resources
You are permitted to use the following during the examination:

- Blackboard
- Digital notes stored on OneDrive
- Handwritten notes

Please be reminded that AI tools, online search engines, and any other external digital resources are strictly prohibited.

Materials Covered
Lecture and laboratory content from Weeks 1 to 5

Question Format
This is a practical skills assessment, consisting of:
Multiple Choice Questions (MCQs)
Text-based questions
Activity-based (laboratory) questions

Number of Questions
A total of 25 questions
32

Weighting
This examination contributes 40% to your overall module grade.

Duration
90 minutes in total

Venue and time
The examination will take place during your scheduled lab sessions.

Concepts

- **Conversion Table:**

A	B	C	D	E	F	G	H	I	J	K	L	M
↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕
0	1	2	3	4	5	6	7	8	9	10	11	12

N	O	P	Q	R	S	T	U	V	W	X	Y	Z
↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕
13	14	15	16	17	18	19	20	21	22	23	24	25

- **Modular arithmetic operations:**

Mod refers to the remainder of a division operation. The (mod *n*) operation maps all integers into the set of integers {0, 1, … , (*n* -1)}. The Modular arithmetic exhibits the following properties:

1. $[(a \bmod n) + (b \bmod n)] \bmod n = (a + b) \bmod n$
2. $[(a \bmod n) - (b \bmod n)] \bmod n = (a - b) \bmod n$
3. $[(a \bmod n) \times (b \bmod n)] \bmod n = (a \times b) \bmod n$

Examples:

(11 + 15) mod 8 = 26 mod 8 = 2
[(11 mod 8) + (15 mod 8)] mod 8 = [3 + 7] mod 8 = 10 mod 8 = 2

(11 - 15) mod 8 = - 4 mod 8 = 4
[(11 mod 8) - (15 mod 8)] mod 8 = [3 - 7] mod 8 = - 4 mod 8 = 4

(11 × 15) mod 8 = 165 mod 8 = 5
[(11 mod 8) × (15 mod 8)] mod 8 = [3 × 7] mod 8 = 21 mod 8 = 5

What is a matrix?

A Matrix is just rectangular arrays of items
A typical matrix is a rectangular array of numbers arranged in rows and columns.

$$\underset{3x4}{A} = \begin{bmatrix} 21 & 62 & 33 & 93 \\ 44 & 95 & 66 & 13 \\ 77 & 38 & 79 & 33 \end{bmatrix}$$

Sizing a matrix

By convention matrices are “sized” using the number of rows (m) by number of columns (n).

$$\underset{3x4}{A} = \begin{bmatrix} 21 & 62 & 33 & 93 \\ 44 & 95 & 66 & 13 \\ 77 & 38 & 79 & 33 \end{bmatrix} \qquad \underset{3x3}{B} = \begin{bmatrix} 7 & 3 & 2 \\ 8 & 4 & 1 \\ 6 & 5 & 9 \end{bmatrix}$$

$$\underset{4x2}{C} = \begin{bmatrix} 11 & 4 \\ 14 & 7 \\ 16 & 8 \\ 22 & 3 \end{bmatrix} \qquad \underset{1x1}{D} = \begin{bmatrix} 17 \end{bmatrix}$$

“Special” Matrices

Square matrix: a square matrix is an mxn matrix in which m = n.

$$\underset{3x3}{B} = \begin{bmatrix} 7 & 3 & 2 \\ 8 & 4 & 1 \\ 6 & 5 & 9 \end{bmatrix}$$

Vector: a vector is an mxn matrix where either m OR n = 1 (but not both).

$$\underset{4x1}{X} = \begin{bmatrix} 12 \\ 9 \\ -4 \\ 0 \end{bmatrix} \qquad \underset{1x3}{Y} = \begin{bmatrix} 7 & -22 & 14 \end{bmatrix}$$

“Special” Matrices

Scalar: a scalar is an mxn matrix where BOTH m and n = 1.

$$\underset{1x1}{D} = [17]$$

Zero matrix: an mxn matrix of zeros.

$$\underset{3x2}{0} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$

$$\underset{3x3}{I} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Identity Matrix: a square (mxm) matrix with 1s on the diagonal and zeros everywhere else.

Matrix Multiplication

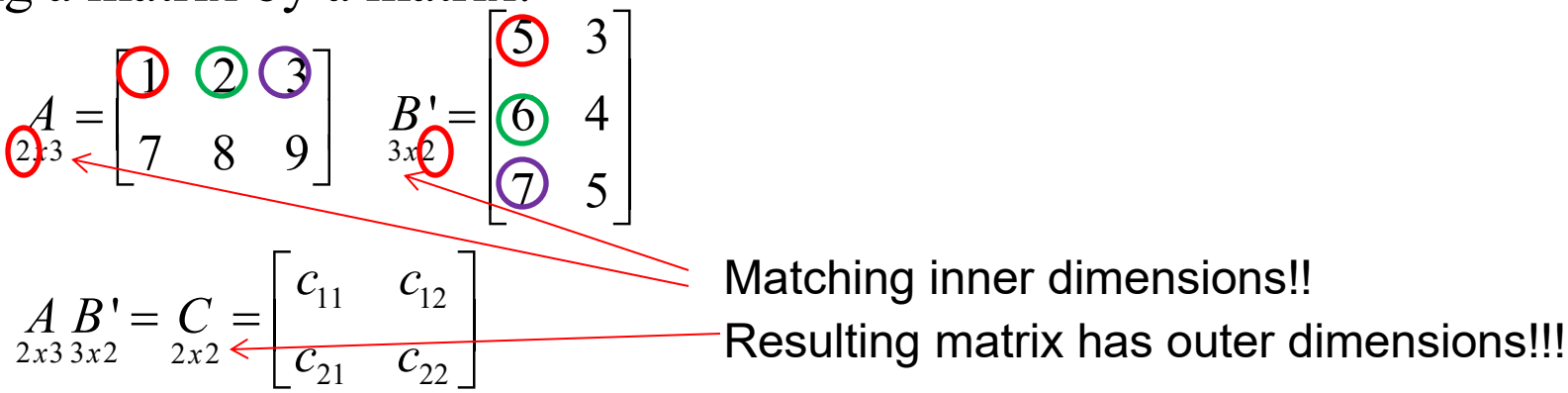
Multiplying a matrix by a scalar: each element in the matrix is multiplied by the scalar.

$$\underset{2x3}{A} = \begin{bmatrix} 1 & 2 & 3 \\ 7 & 8 & 9 \end{bmatrix} \quad \text{and} \quad \underset{1x1}{x} = 5; \text{ then}$$

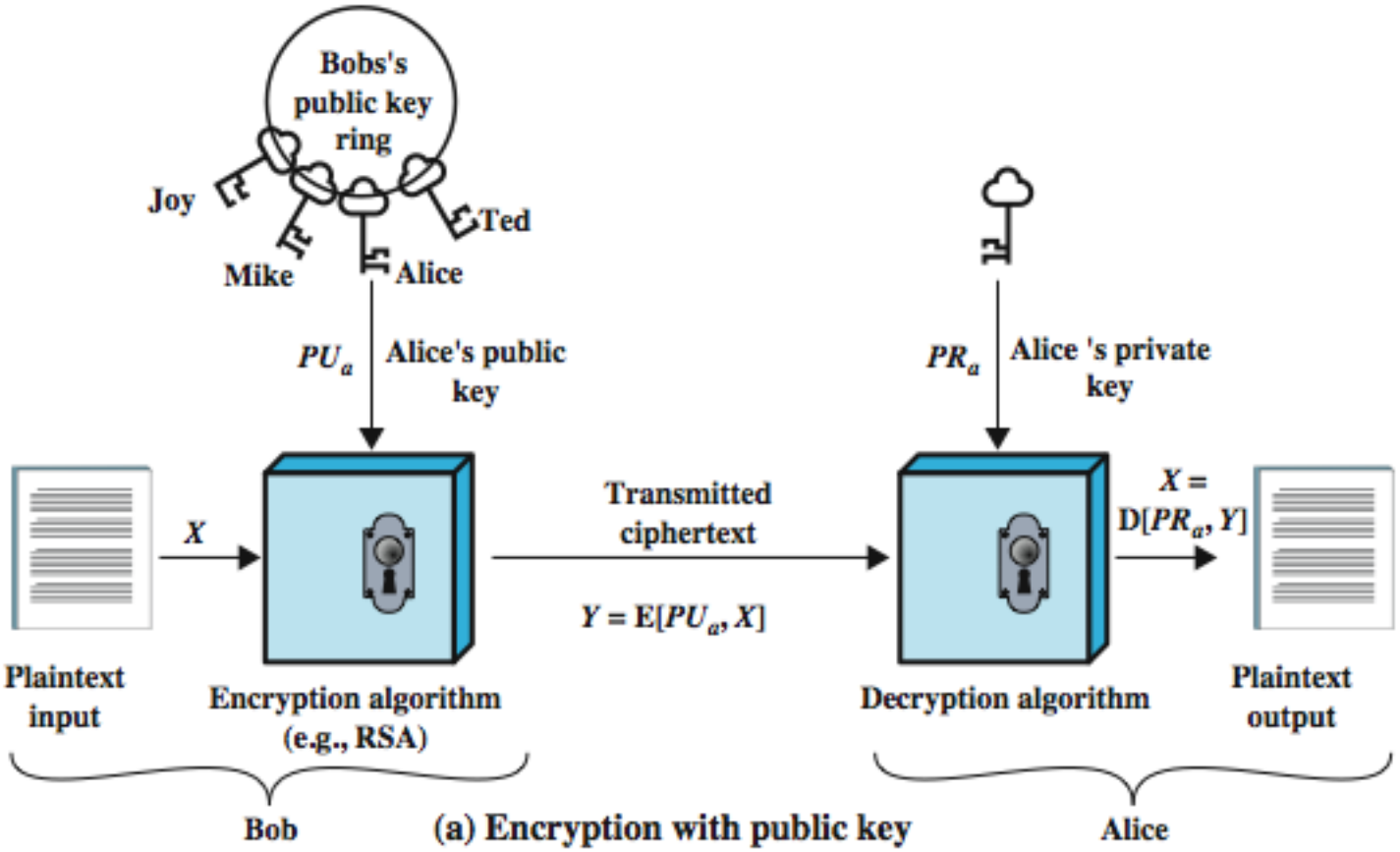
$$xA = \begin{bmatrix} 5*1 & 5*2 & 5*3 \\ 5*7 & 5*8 & 5*9 \end{bmatrix} = \begin{bmatrix} 5 & 10 & 15 \\ 35 & 40 & 45 \end{bmatrix}$$

Matrix Multiplication

Multiplying a matrix by a matrix:



Public-Key Cryptography



(a) Encryption with public key

Public-Key Cryptography

In public key cryptography, if we know:

- the encryption algorithm **and**
- the key

to determine the ciphertext then how is it possible that we cannot work out what the plaintext (decryption key) is from this information?

One-way functions

A **one-way function** is “easy” to compute and “difficult” to reverse.
It is easy to take two prime numbers and multiply them together.
If the numbers are small, we can do this in our heads, on a piece of paper.
What if numbers get bigger and bigger?
Multiplication of two prime numbers is **believed** to be a one-way function.
The process of **exponentiation** means raising numbers to a power.

Raising **2** to the power **3**, normally denoted 2^3 just means multiplying **2** by itself **3** times. In other words:

- 2^3 =(2x2x2)
- a^b = a x a x a x … x a

Modular exponentiation means computing a^b mod some other number **n**.

- a^b mod n.

In other words, given a number **a** and a prime number **n**, the function

- f(b) = a^b mod n

  Is a one-way function

Required Concepts

Factor
Numbers you can multiply to get the original number
Factors of 12: 1, 2, 3, 4, 6, 12
Factors of 7: 1, 7

Prime
A number whose factors are only 1 and itself.
Prime Numbers: 2, 3, 5, 7, 11, 13, 29, 37, 61
Number divisible by only 1 and itself.

Semi-Prime
Numbers whose factors are prime numbers.
Factors of Semi-Prime 21: 1, 3, 7, 21
Product of two prime numbers is always a semi-prime.

Modulo
Remainder division.
13 MOD 5 = 3
21 MOD 5 = 1
25 MOD 5 = 0

RSA

The **RSA** algorithm was the first practical implementation of public key encryption.
It is named after the three researchers Ron Rivest, Adi Shamir and Len Adleman
Let **n** be the product of two large primes **p** and **q** **(i.e. n=pxq)**

- at least 512 bits.

Select a special number **e**

- greater than 1 and less than (**p**-1)(**q**-1).
- The precise mathematical property that **e** must have is that there must be no numbers that divide neatly into **e** and into (**p**-1)(**q**-1), except for 1.

Publish the pair of numbers (**n**,**e**)
Compute the private key **d** from **p**, **q** and **e**

The private key **d** is computed to be the unique inverse of **e** modulo (**p**-1)(**q**-1).

In other words, **d** is the unique number less than (**p**-1)(**q**-1) that when multiplied by **e** gives you 1 modulo (**p**-1)(**q**-1).

- ed = 1 mod (p-1)(q-1)

Public key: (e, n)
Private key: d

RSA Example

Prime Numbers: P=7, Q=19
Product (N): 133
Totient: 108
Public Key (E): 29
Private Key (D): 41

- **Generating Keys:**
  - **Select two Prime Numbers**: P , Q
  - **Calculate Product**: N=P×Q
  - **Calculate Totient**: Totient=(P−1)×(Q−1)

- **Select Public Key**: (E)
  - Must be prime
  - Must be less than the Totient
  - Must **NOT** be a factor of the Totient

- **Select a Private Key**: (D)
  - The product of (D) and (E), divided by the Totient, must result in a remainder of 1:
    $(D \times E) \bmod \text{Totient} = 1$

RSA Example

Prime Numbers: P=7, Q=19
Product (N): 133
Totient: 108
Public Key (E): 29
Private Key (D): 41

- Encryption and Decryption:
  - Encryption:

    Message^E MOD N = *Cipher Text*
  - Decryption:

    Cipher^D MOD N = *Message*

Message
60

- **Encrypt** with Public Key, **Decrypt** with Private Key

(60^29) MOD 133 = 86

(86^41) MOD 133 = 60

RSA Example

Prime Numbers: P=7, Q=19
Product (N): 133
Totient: 108
Public Key (E): 29
Private Key (D): 41

Message
60

- Encryption and Decryption:
  - Encryption:

    Message^D MOD N = *Cipher Text*
  - Decryption:

    Cipher^E MOD N = *Message*

- **Encrypt** with Private Key, **Decrypt** with Public Key

(60^41) MOD 133 = 72

(72^29) MOD 133 = 60

RSA

- How secure is RSA?
  - Security lies in difficulty of factoring Semi Prime numbers
    - If given the number 133, could you extract 7 and 19?
      - Really? … Prove it, what are the factors of 1909?

Only 11 Bits

  - In 1991, RSA Laboratory created the RSA Challenge:
    - Released 54 Semi-Primes of various sizes and asked for Factors
      - Competition ended in 2007 – only 12 factors were identified
      - As of 2020, another 11 were identified (no cash was awarded)
        - Biggest number factored: 829 Bits (Feb 2020)
      - In 29 years, the 1024 bit number has never been factored
        - 1024 bit RSA keys was recommended standard since 2002
        - 2048 bit RSA Keys is recommended standard since 2015

RSA

- How secure is RSA?

1024 bit Semi-Prime number:

```
135066410865995223349603216278805969938881475605667027524485143851526510604859533833940287150571909441798207282164471551373680419703964191743046496589274256239341020864383202110372958725762358509643110564073501508187510676594629205563685529475213500852879416377328533906109750544334999811150056977236890927563
```

- 1024 bit RSA keys was recommended standard since 2002
- 2048 bit RSA Keys is recommended standard since 2015

Diffie–Hellman (DH) key exchange

- The **Diffie–Hellman (DH) key exchange** technique was first defined in their seminal paper in 1976.
- DH key exchange has the following important properties:
  - The resulting shared secret cannot be computed by either of the parties without the cooperation of the other.
  - A third party observing all the messages transmitted during DH key exchange cannot deduce the resulting shared secret at the end of the protocol.

Principle behind DH

- Assume that Alice and Bob are the parties who wish to establish a shared secret, and let their public and private keys in the public key cipher system be denoted by (PA , SA) and (PB , SB) respectively.
- The basic principle behind Diffie–Hellman key exchange is as follows:

1. Alice and Bob exchange their public keys PA and PB.
2. Alice computes F(SA , PB)

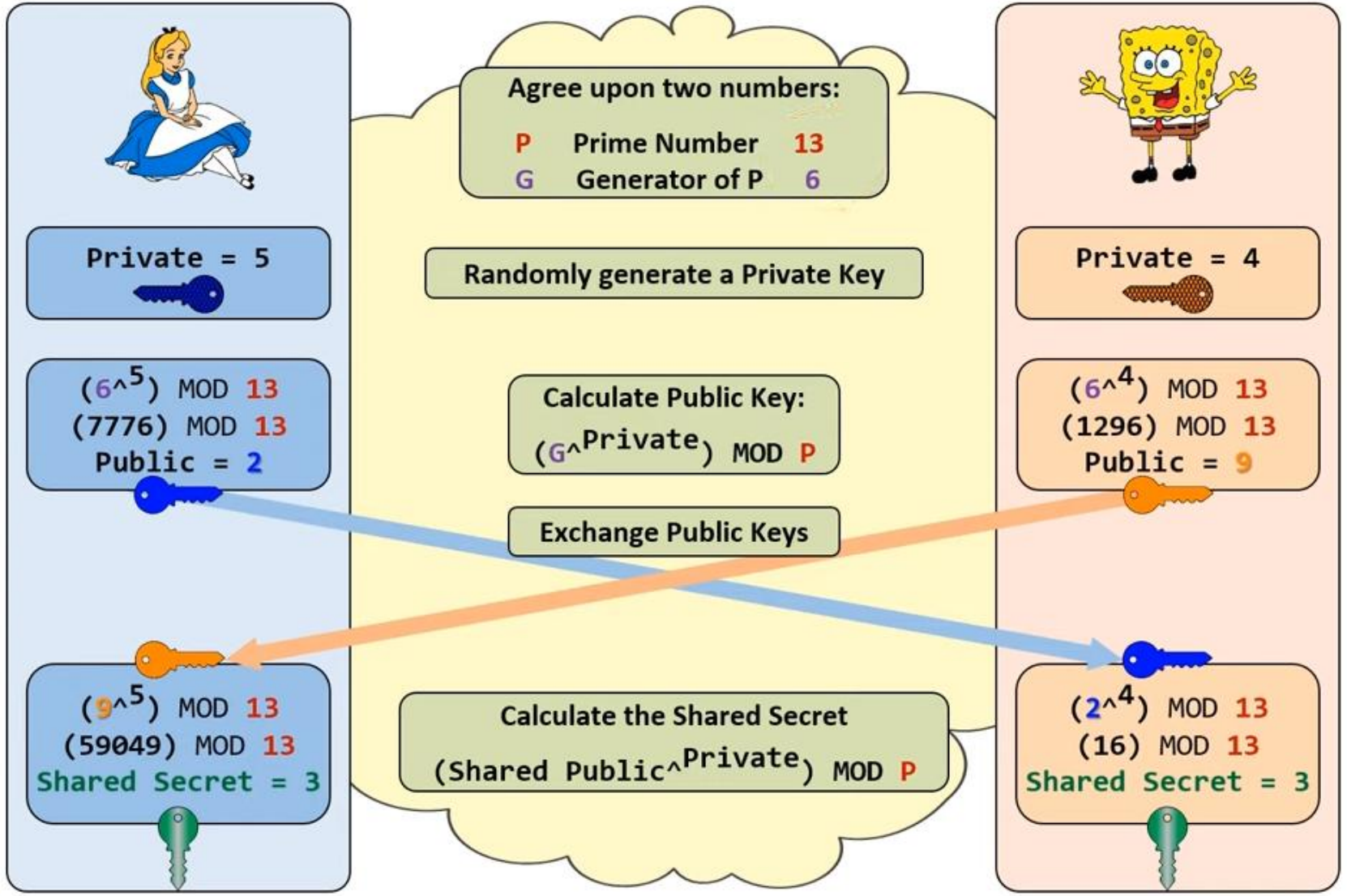
$$K = (PB)^{S_A} \bmod q$$

3. Bob computes F(SB, PA)

$$K = (PA)^{S_B} \bmod q$$

4. The special property of the public key cipher system, and the choice of the function F, are such that F(SA , PB) = F(SB, PA). If this is the case then Alice and Bob now share a secret.
5. This shared secret can easily be converted by some public means into a bitstring suitable for use as, for example, a DES key.

Principle behind DH



Trust in Encryption

Cryptanalysis

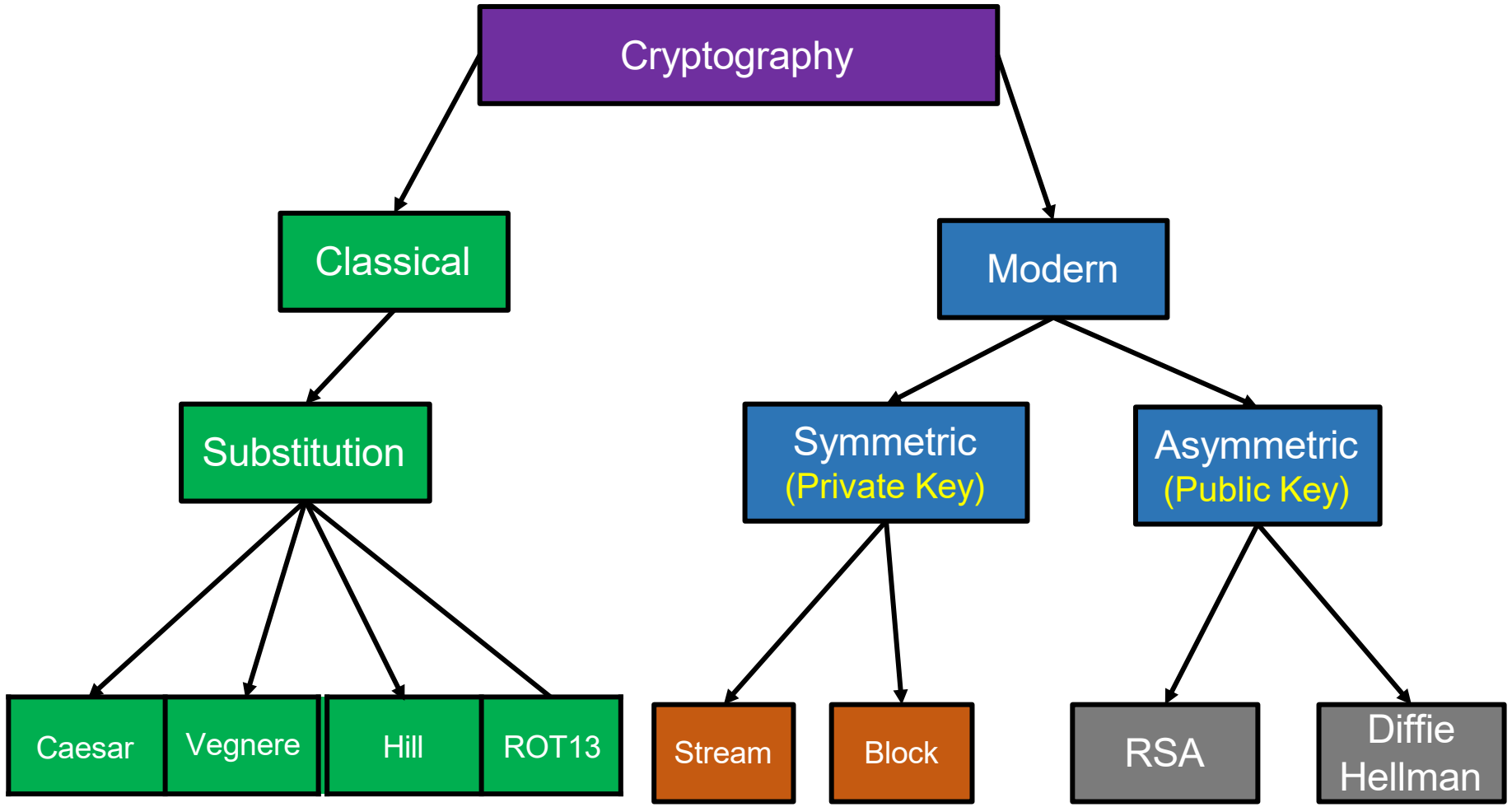
Type of Attack	Known to Cryptanalyst
Ciphertext only	•Encryption algorithm •Ciphertext to be decoded
Known plaintext	•Encryption algorithm •Ciphertext to be decoded •One or more plaintext-ciphertext pairs formed with the secret key
Chosen plaintext	•Encryption algorithm •Ciphertext to be decoded •Plaintext message chosen by cryptanalyst, together with its corresponding ciphertext generated with the secret key

Trust in Encryption

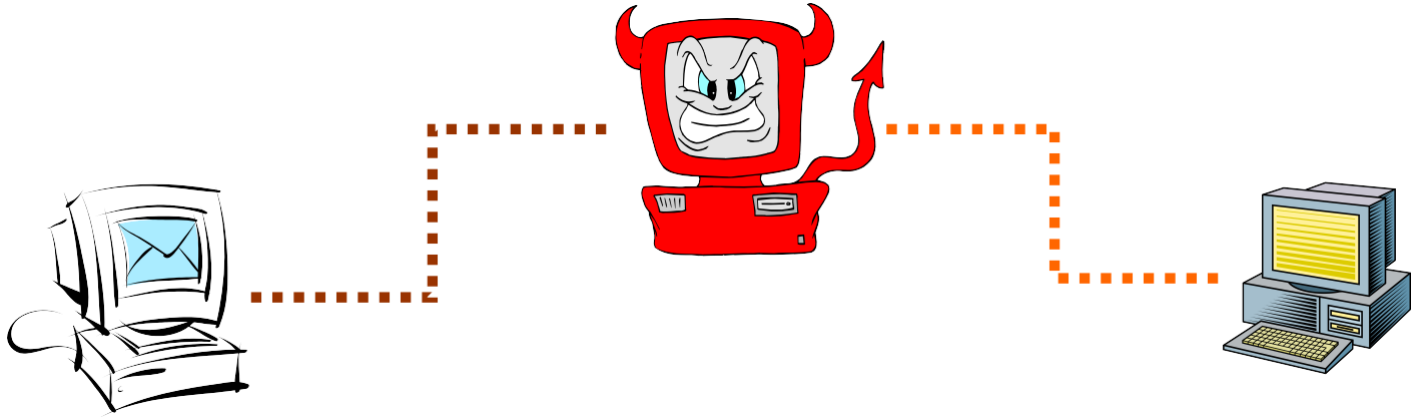
Cryptanalysis

Type of Attack	Known to Cryptanalyst
Chosen ciphertext	•Encryption algorithm •Ciphertext to be decoded •Purported ciphertext chosen by cryptanalyst, together with its corresponding decrypted plaintext generated with the secret key
Chosen text	•Encryption algorithm •Ciphertext to be decoded •Plaintext message chosen by cryptanalyst, together with its corresponding ciphertext generated with the secret key •Purported ciphertext chosen by cryptanalyst, together with its corresponding decrypted plaintext generated with the secret key

What we have covered



Data Integrity and Source Authentication



- Encryption does not protect data from modification by another party.
  - Why?
- Need a way to ensure that data arrives at destination in its original form as sent by the sender and it is coming from an authenticated source.

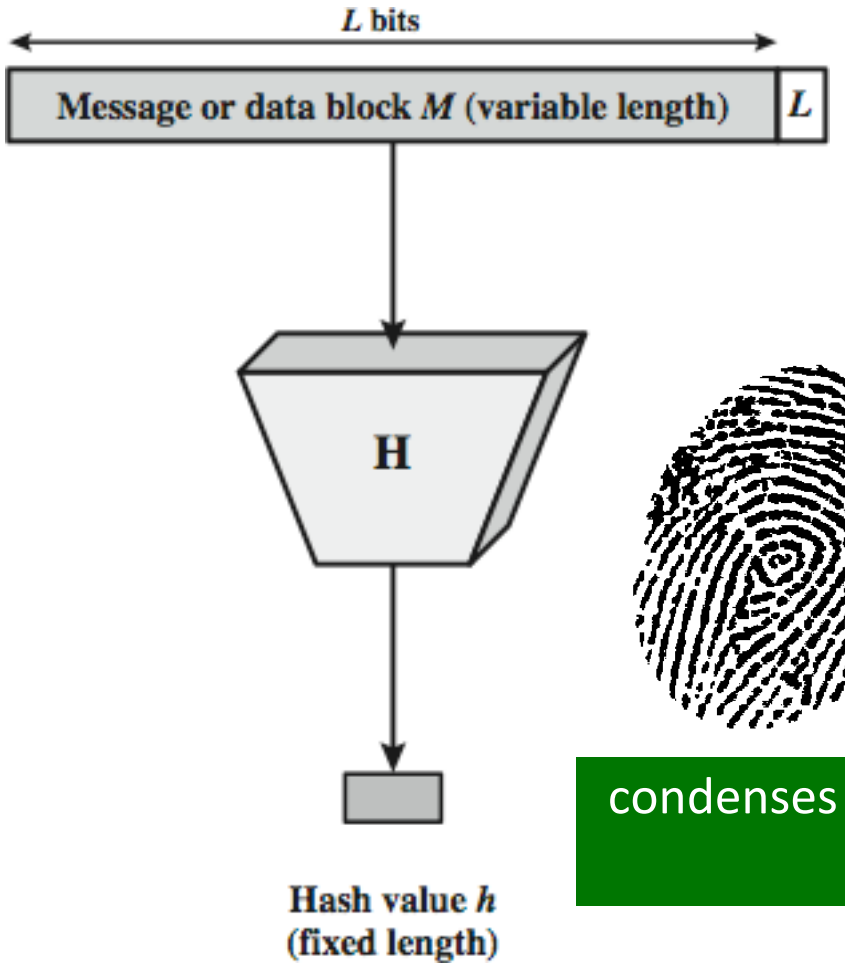
Hash Functions

- A hash function maps a message of an arbitrary length to a m-bit output
  - output known as the fingerprint or the message digest

- What is an example of hash functions?
  - Give a hash function that maps Strings to integers in [0,2^{32}-1]

- Cryptographic hash functions are hash functions with additional security requirements

Hash Functions



- The hash value represents concisely the longer message
  - may called the *message digest*

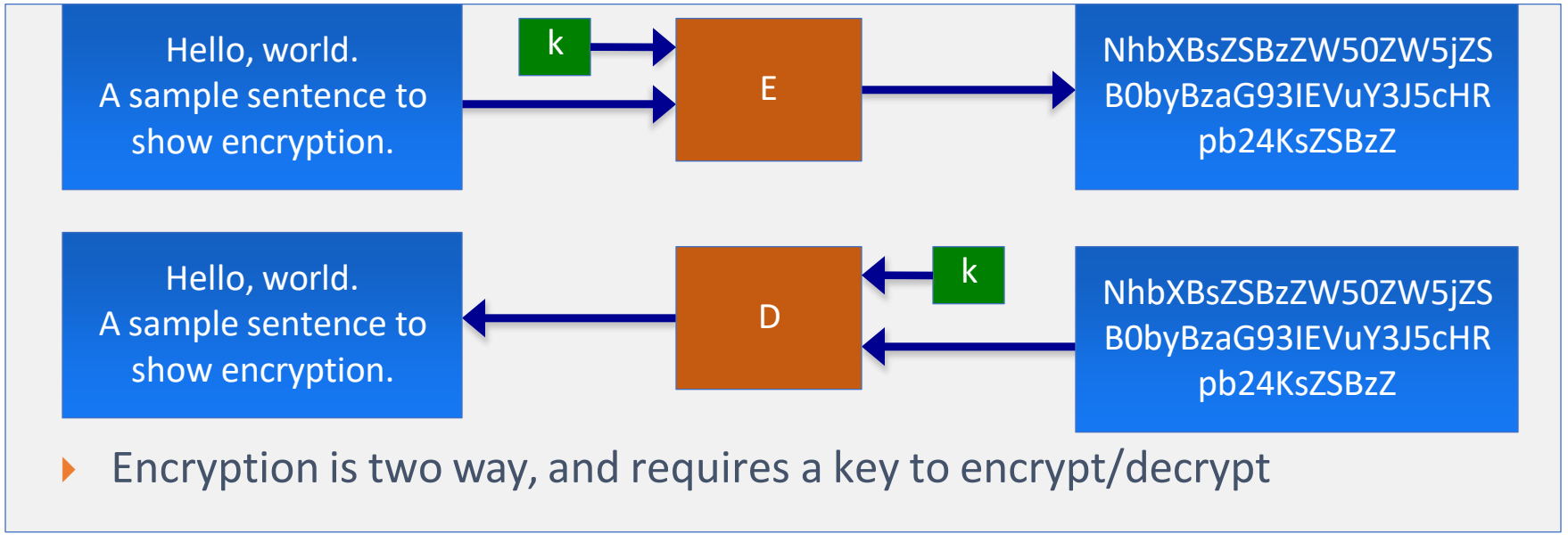


- A message digest is as a ``digital fingerprint'' of the original document

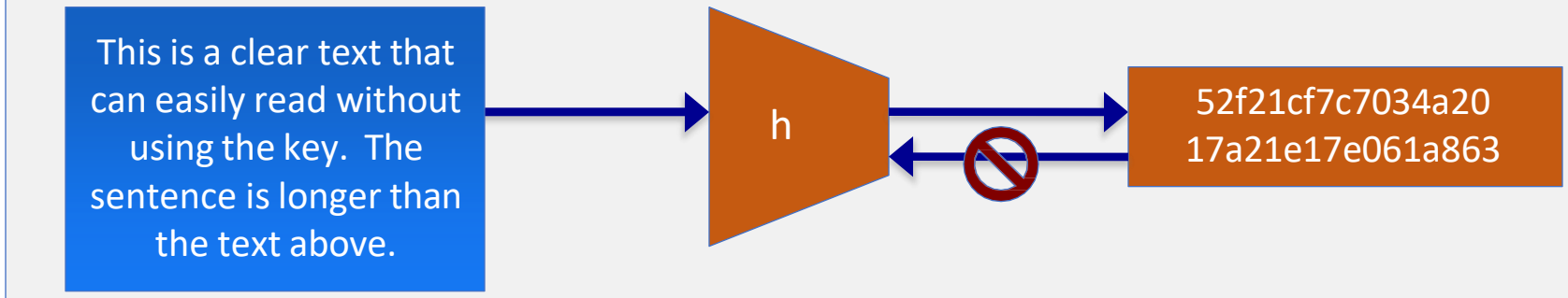
condenses arbitrary message to fixed size

$$h = H(M)$$

Hashing V.S. Encryption



- Encryption is two way, and requires a key to encrypt/decrypt



- Hashing is one-way. There is no 'de-hashing'

Hash Family

- A hash family is a four-tuple (X,Y,K,H), where
  - X is a set of possible messages
  - Y is a finite set of possible message digests
  - K is the keyspace
  - For each $K \in K$, there is a hash function $h_K \in H$. Each h_K: $X \rightarrow Y$
- Alternatively, one can think of H as a function $K \times X \rightarrow Y$

Hash Functions Family

- MD (Message Digest)
  - Designed by Ron Rivest
  - Family: MD2, MD4, MD5
- SHA (Secure Hash Algorithm)
  - Designed by NIST
  - Family: SHA-0, SHA-1, and SHA-2
    - SHA-2: SHA-224, SHA-256, SHA-384, SHA-512
    - SHA-3: New standard in competition
- RIPEMD (Race Integrity Primitive Evaluation Message Digest)
  - Developed by Katholieke University Leuven Team
  - Family : RIPEMD-128, RIPEMD-160, RIPEMD-256, RIPEMD-320

MD5, SHA-1, and RIPEMD-160

	MD5	SHA-1	RIPEMD-160
Digest length	128 bits	160 bits	160 bits
Basic unit of processing	512 bits	512 bits	512 bits
Number of steps	64 (4 rounds of 16)	80 (4 rounds of 20)	160 (5 paired rounds of 16)
Maximum message size	∞	$2^{64} - 1$ bits	$2^{64} - 1$ bits
Primitive logical functions	4	4	5
Additive constants used	64	4	9
Endianness	Little-endian	Big-endian	Little-endian

Cryptographic Hash Function Properties

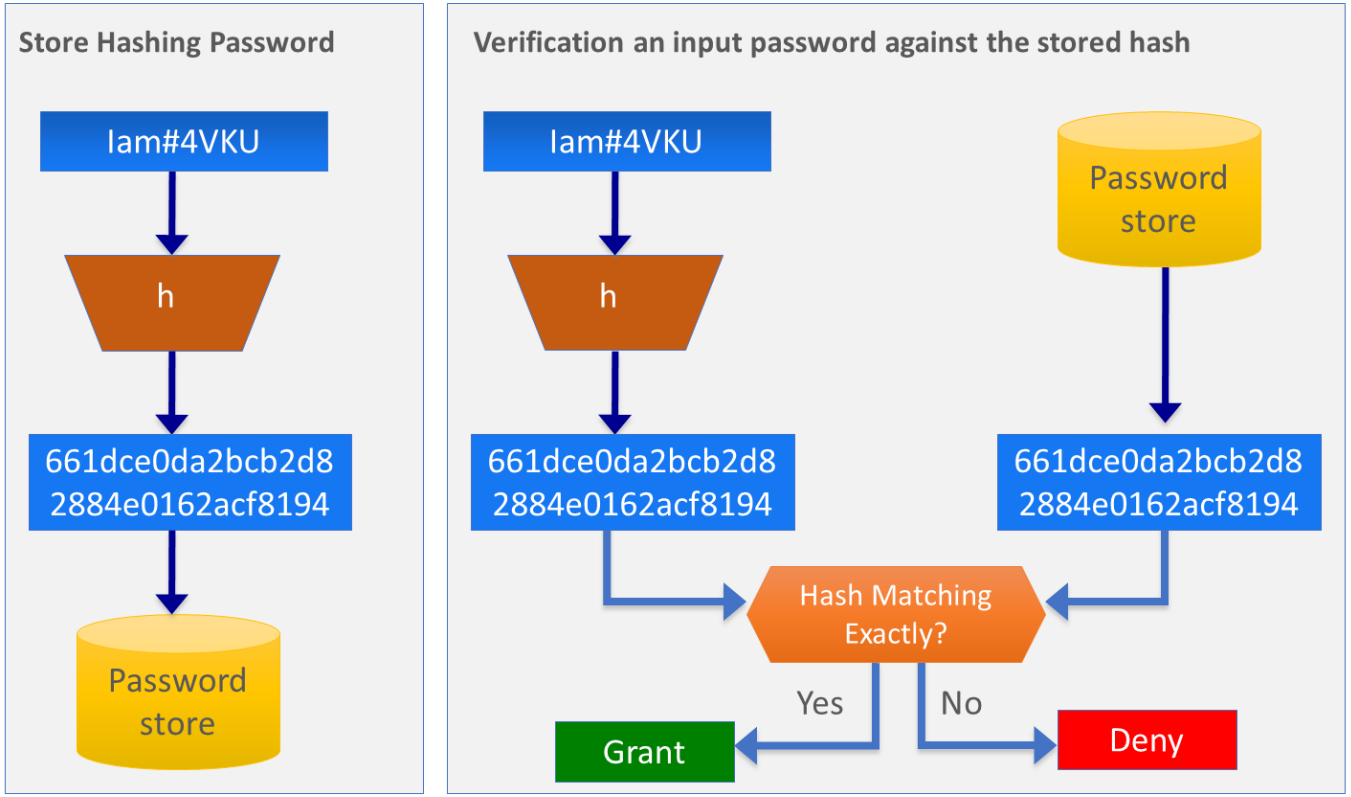
- There are several requirements the hash function has to meet to be considered secure.
- *Speed*
  - *That means the hashing function should be able to produce a hash in a fraction of a second.*
- Avalanche Effect
  - *Hi 3639efcd08abb273b1619e82e78c29a7df02c1051b1820e99fc395dcaa3326b8*
  - *hi 8f434346648f6b96df89dda901c5176b10a6d83961dd3c1ac88b59b2dc327aa4*
- Hash Function Should Be Deterministic
  - Same input, will always result in the same output
- Pre-Image Resistance (One-Way Function)
  - Can't reverse the cryptographic hash function
- Collision Resistance
  - *two different messages shouldn't be able to produce the same hash value.*

Usages of Cryptographic Hash Functions

- Software integrity
  - E.g., tripwire
- Timestamping
  - How to prove that you have discovered a secret on an earlier date without disclosing it?
- Covered later
  - Message authentication
  - One-time passwords
  - Digital signature

Examples of Cryptographic Hash Functions

- Password Verification
  - Nearly all operating systems and websites store passwords in hashes



Examples of Cryptographic Hash Functions

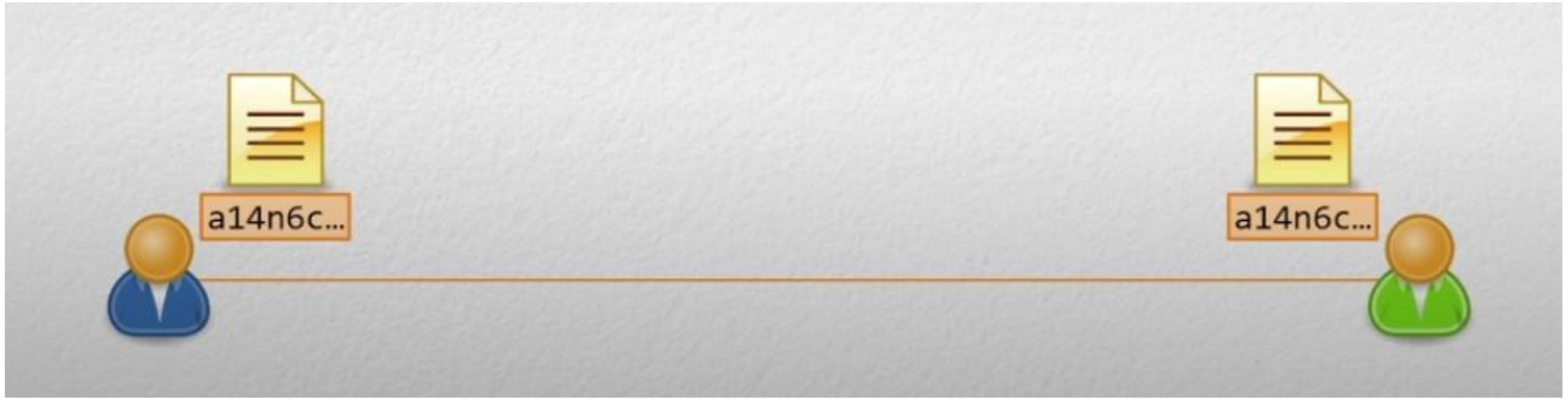
- Cryptocurrencies
  - Cryptographic hash functions are widely used in cryptocurrencies to pass transaction information anonymously.
  - For example, Bitcoin, the original and largest cryptocurrency, uses the SHA-256 cryptographic hash function in its algorithm.
- Signature Generation and Verification
  - Verifying signatures is a mathematical process used to verify the authenticity of digital documents or messages
- Verifying File and Message Integrity

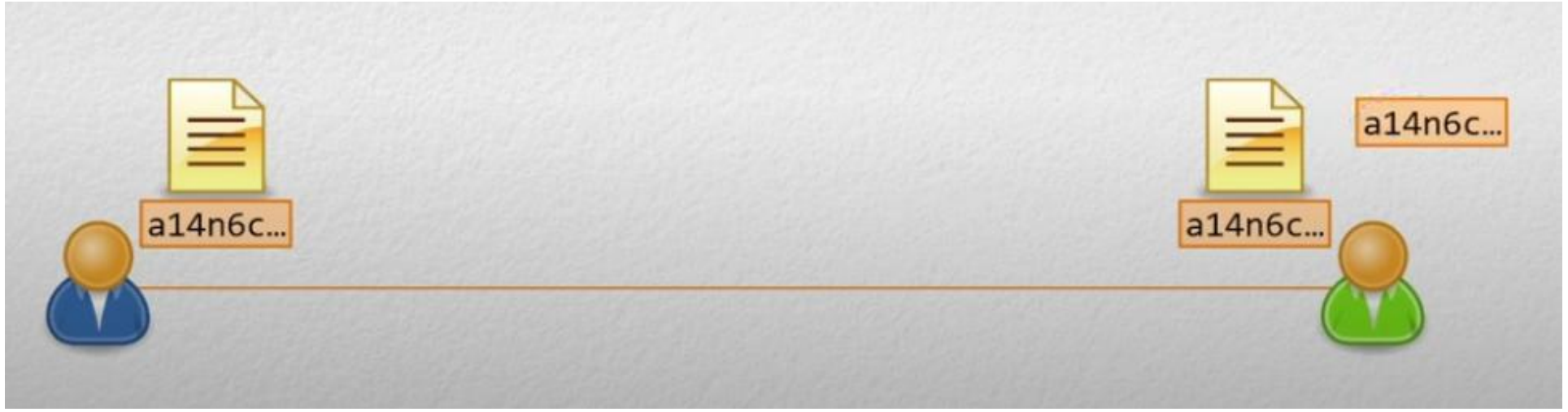
Using Hash Functions for Message Integrity

- Uses a Hash Function h, assuming an authentic (adversary cannot modify) channel for short messages
  - Transmit a message M over the normal (insecure) channel
  - Transmit the message digest h(M) over the secure channel
  - When receiver receives both M' and h, how does the receiver check to make sure the message has not been modified?

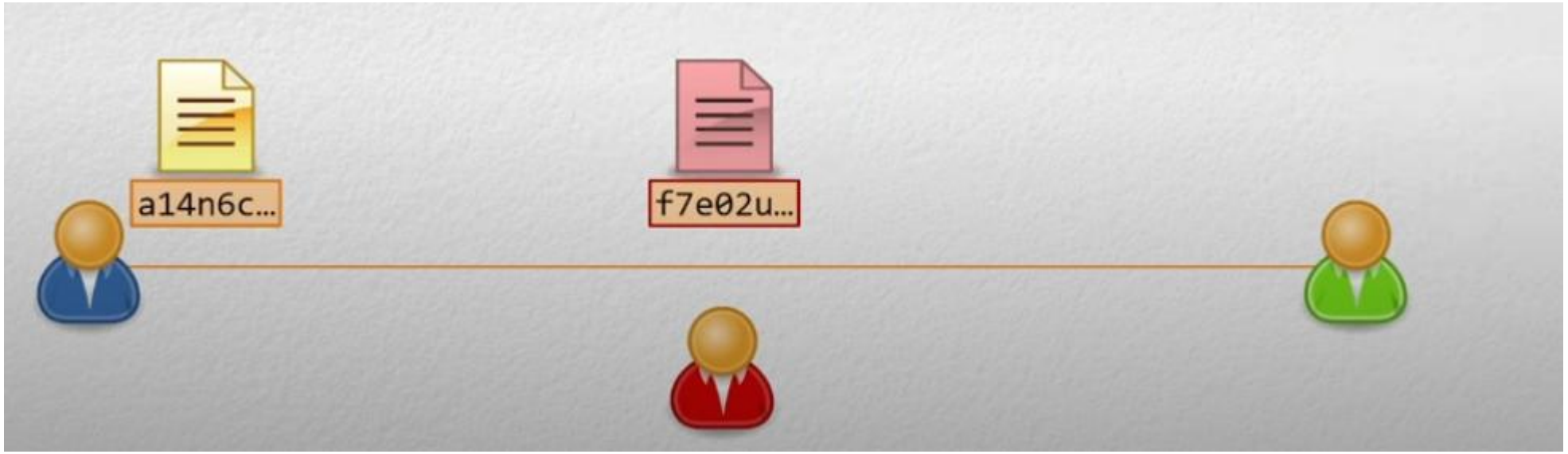
- This is insecure. How to attack it?
- A hash function is a many-to-one function, so collisions can happen.

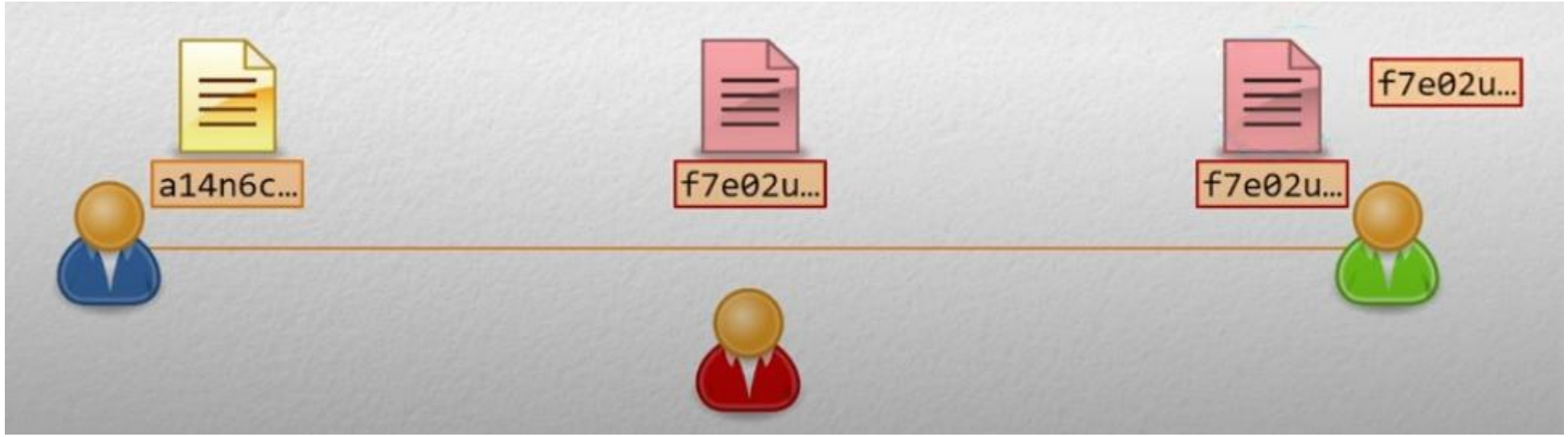
Example



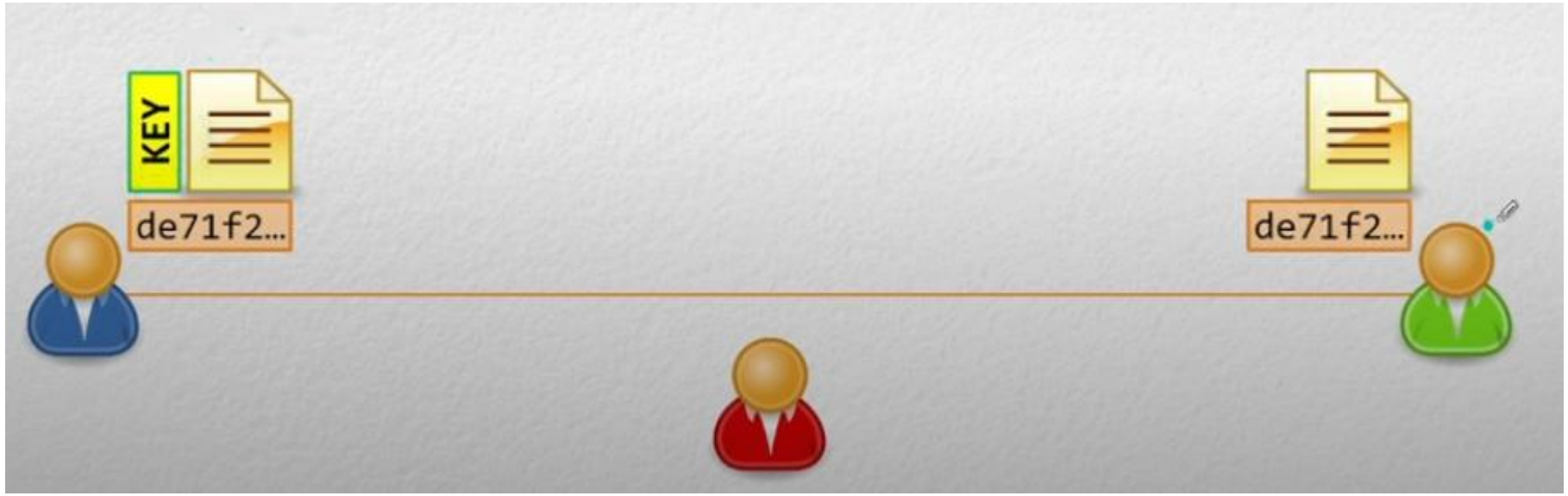


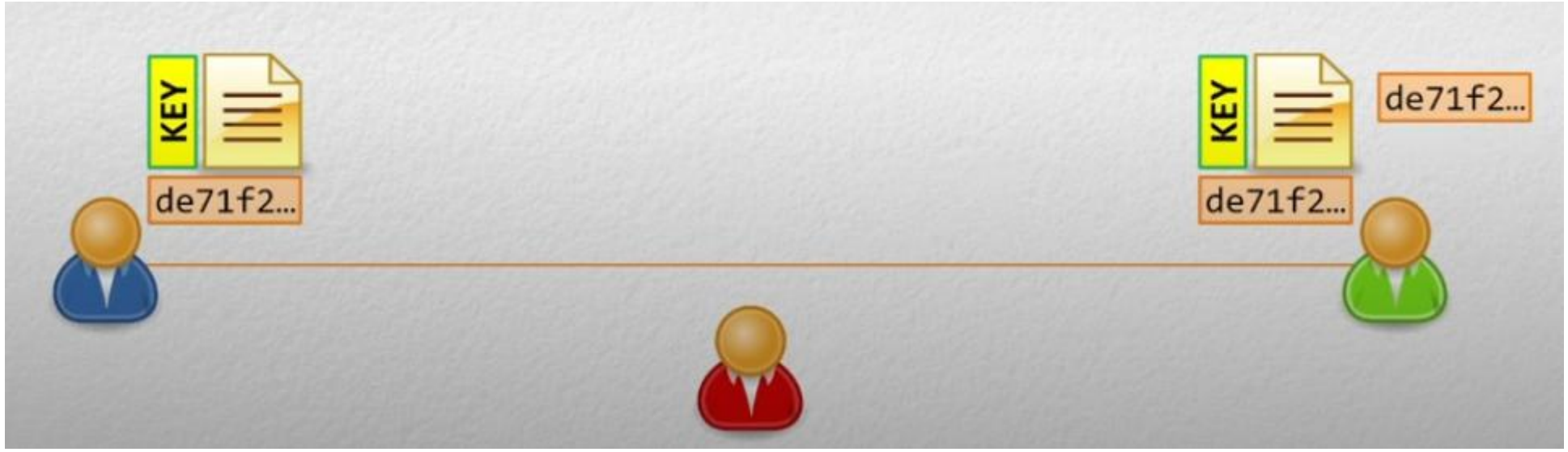
Example



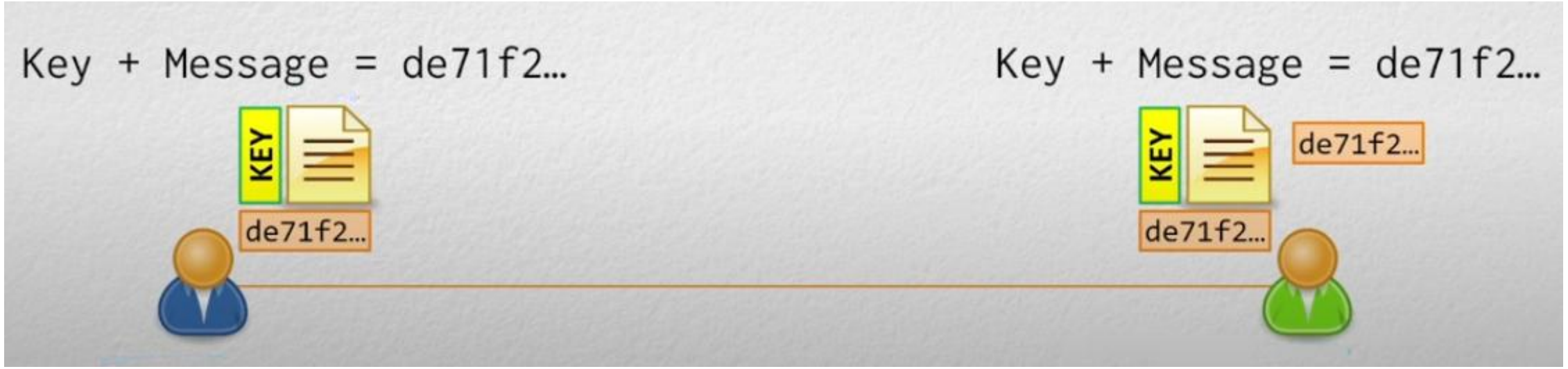


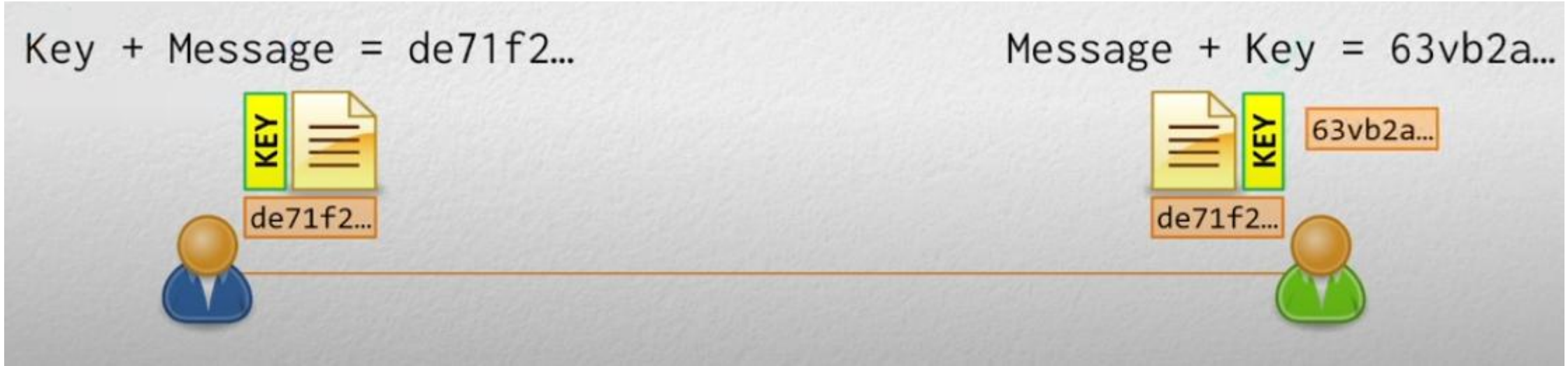
Example





Example


Key + Message = de71f2…
KEY
de71f2…
Key + Message = de71f2…
KEY
de71f2…
de71f2…


Key + Message = de71f2…
KEY
de71f2…
Message + Key = 63vb2a…
KEY
63vb2a…
de71f2…

Message Authentication Code

- A MAC scheme is a hash family, used for message authentication
- MAC(K,M) = $H_K(M)$
- The sender and the receiver share secret K
- The sender sends (M, $H_k(M)$)
- The receiver receives (X,Y) and verifies that $H_K(X)=Y$, if so, then accepts the message as from the sender
- To be secure, an adversary shouldn't be able to come up with (X',Y') such that $H_K(X')=Y'$.

Security Requirements for MAC

- Resist the Existential Forgery under Chosen Plaintext Attack
  - Challenger chooses a random key K
  - Adversary chooses a number of messages M_1, M_2, .., M_n, and obtains t_j=MAC(K,M_j) for $1 \leq j \leq n$
  - Adversary outputs M' and t'
  - Adversary wins if $\forall j$ M'$\neq M_j$, and t'=MAC(K,M')

- Basically, adversary cannot create the MAC for a message for which it hasn't seen an MAC

Constructing MAC from Hash Functions

- Let h be a one-way hash function

- MAC(K,M) = h(K || M), where || denote concatenation
  - Insecure as MAC
  - Because of the Merkle-Damgard construction for hash functions, given M and t=h(K || M), adversary can compute M'=M||Pad(M)||X and t', such that h(K||M') = t'

HMAC: Constructing MAC from Cryptographic Hash Functions

$$HMAC_K[M] = Hash[(K^+ \oplus opad) \,||\, Hash[(K^+ \oplus ipad)||M)]]$$

- K^+ is the key padded (with 0) to B bytes, the input block size of the hash function
- ipad = the byte 0x36 repeated B times
- opad = the byte 0x5C repeated B times.

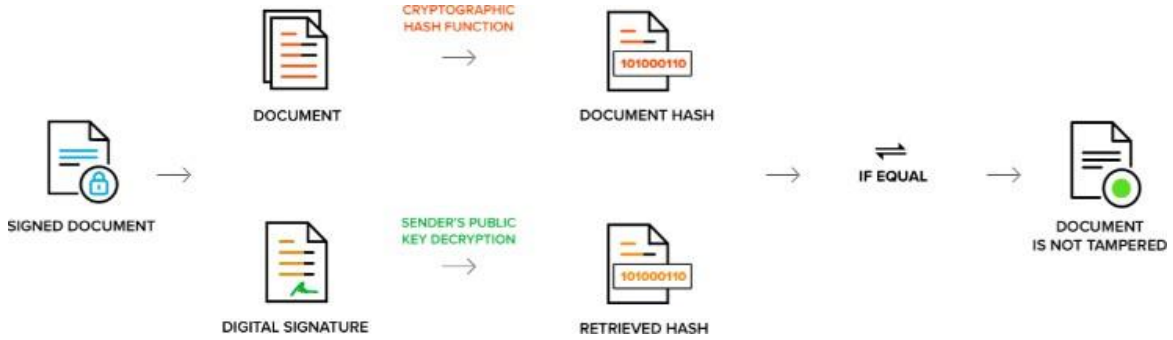
At high level, $HMAC_K[M] = H(K \,||\, H(K \,||\, M))$

Digital Signature



1. Digital Signature is a type of electronic signature that encrypts documents with digital codes that are particularly difficult to duplicate

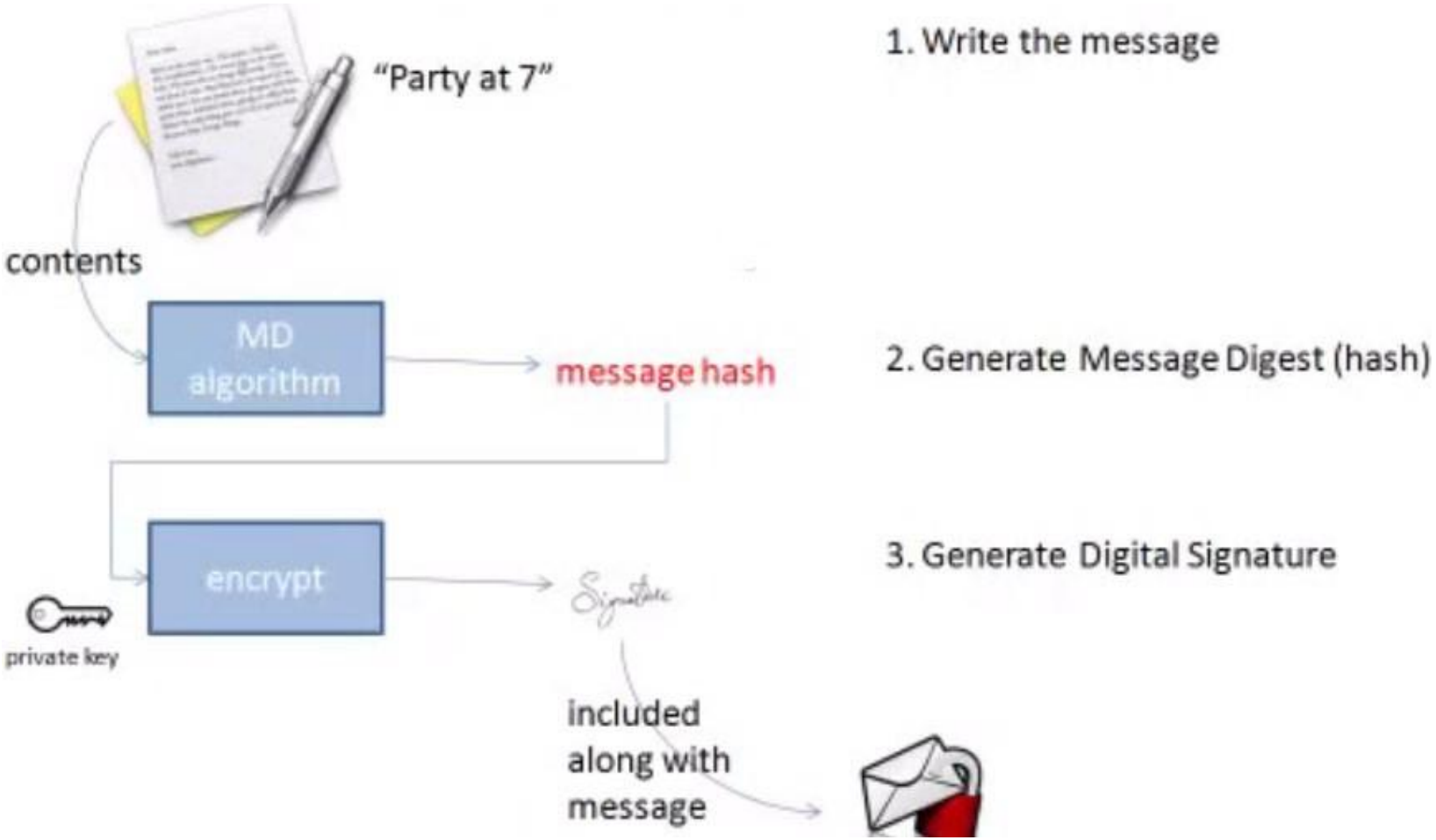
2. In the case of a digital signature message is encrypted with the private key and decrypted with the public key.



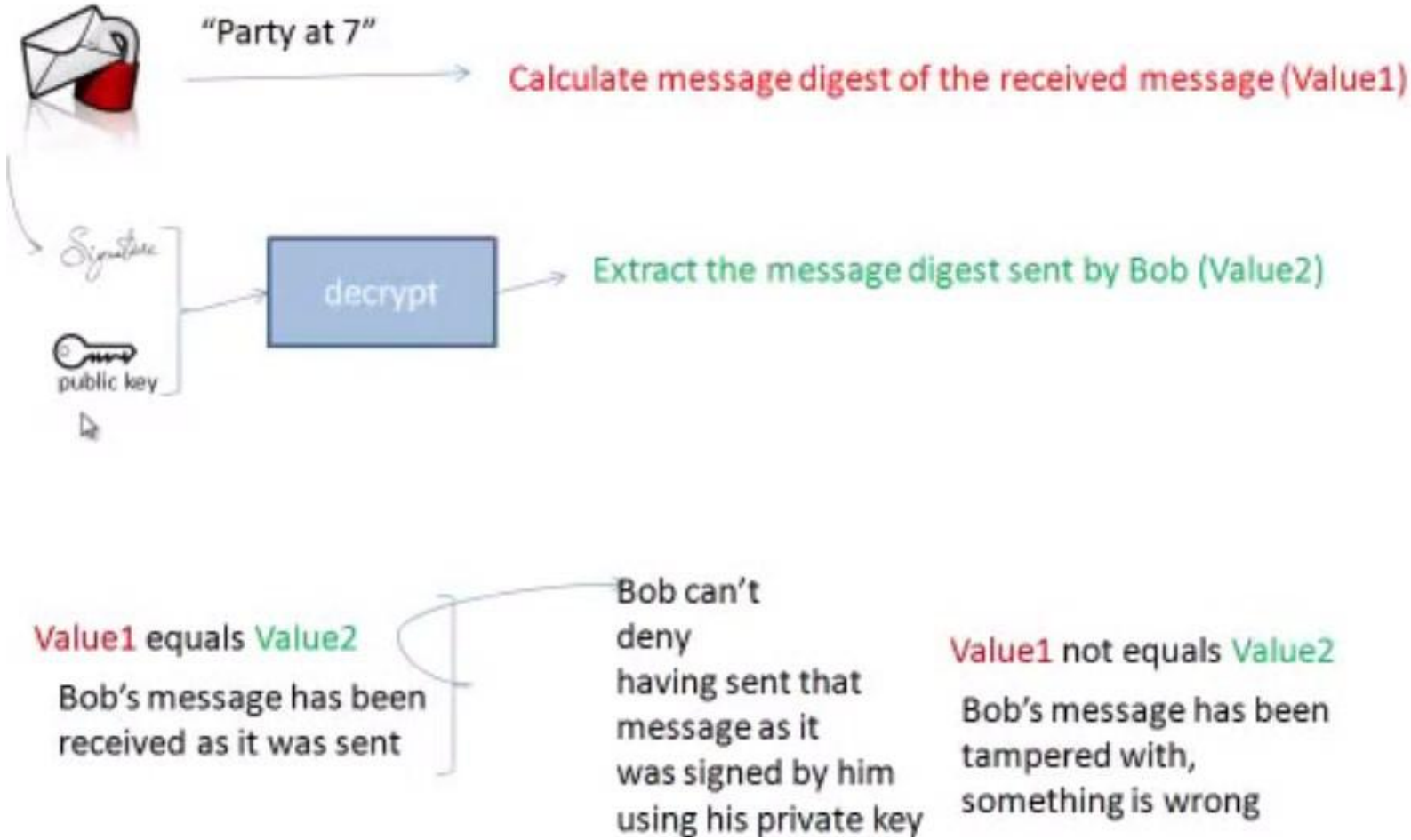
Example

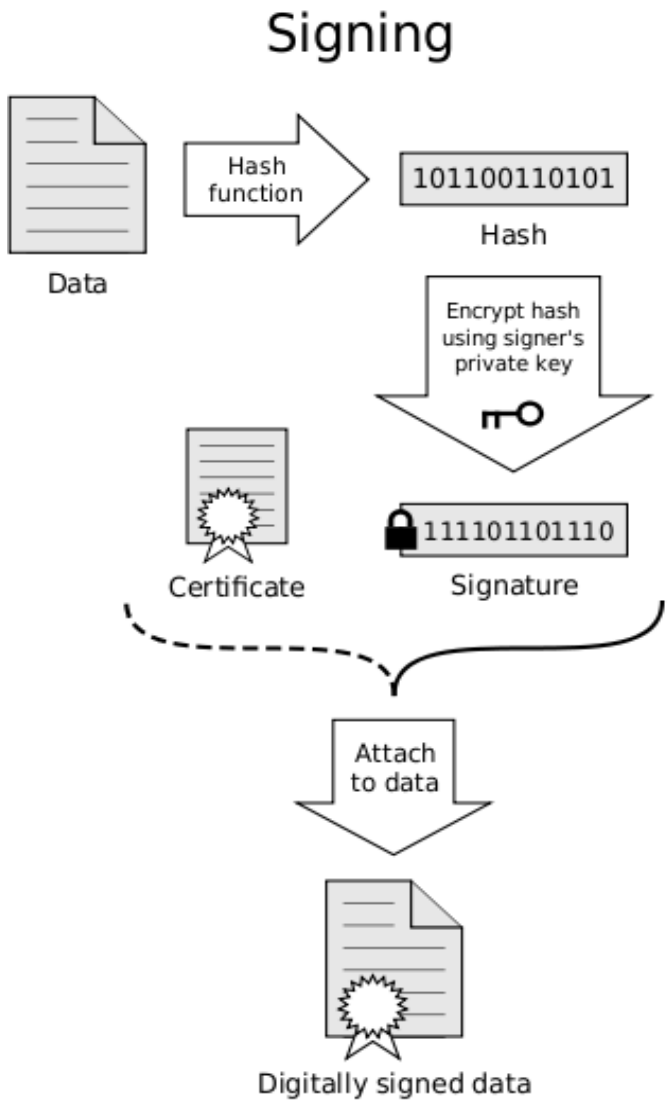

BOB
FRANK
Bob wants to send an important message to Frank
private key
public key
public key
DEVIL

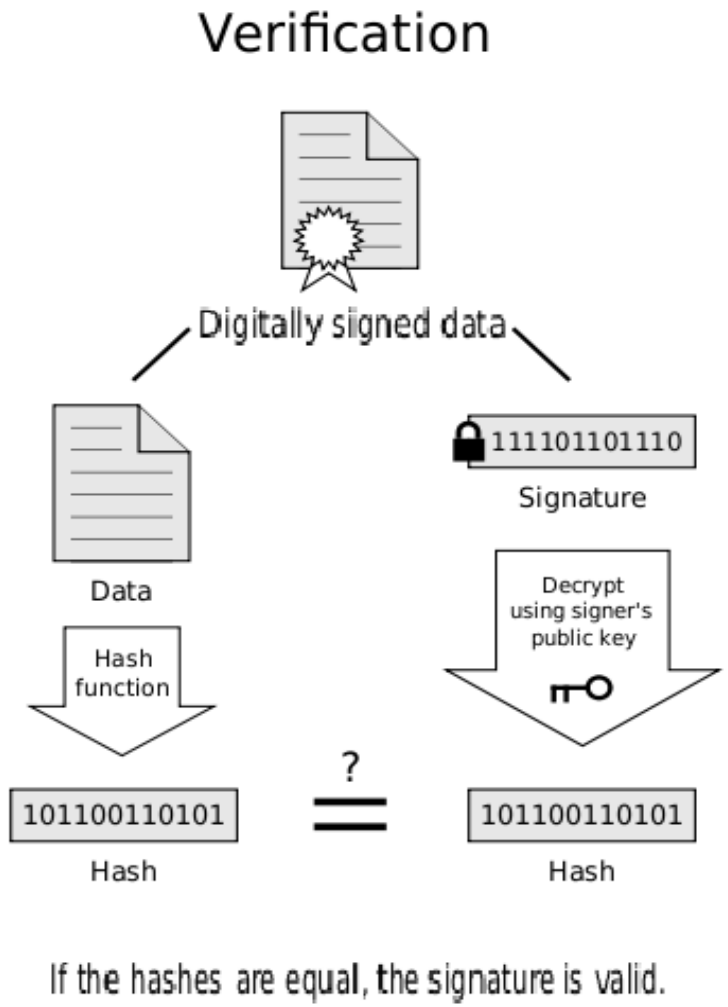
On BOB Side


"Party at 7"
contents
MD algorithm
message hash
encrypt
private key
included along with message
1. Write the message
2. Generate Message Digest (hash)
3. Generate Digital Signature

On FRANK side




Signing
Data
Hash function
101100110101
Hash
Encrypt hash using signer's private key
Certificate
111101101110
Signature
Attach to data
Digitally signed data


Verification
Digitally signed data
Data
111101101110
Signature
Decrypt using signer's public key
Hash function
?
101100110101
Hash
101100110101
Hash
If the hashes are equal, the signature is valid.

Advantages

- Authentication:
  - Identification of person sign.
- Integrity of data:
  - Every change is detected.
- Non Repudiation:
  - Author has encrypted sign on message.
- Speed:
  - Contracts are easily written, completed, and signed by all concerned parties in a little amount of time no matter how far the parties are geographically.

Example

Suppose Alice and Bob are exchanging information and Bob wants to send a signed, but not encrypted message to Alice.

Suppose Bob's RSA setup is as follows:

$$\text{Bob's Public: } E_B(x) = x^5 \pmod{91}$$
$$\text{Bob's Secret: } D_B(y) = y^{29} \pmod{91}$$
$$\text{Bob's Message: } M = 30$$

To securely ***sign*** the message, Bob needs to make a calculation that only he could do. Thus Bob uses his ***secret*** key to sign the message.

$$\text{Bob's signature: } \sigma = D_B(M) = 30^{29} \bmod 91 = 88$$

He sends the message-signature pair to Alice, $(M, \sigma) = (30, 88)$.

Alice can now use Bob's ***public*** information to ***verify*** that this message came from Bob.

$$\text{Alice's verification: } E_B(\sigma) = \sigma^5 = 88^5 \equiv 30 \equiv M \pmod{91}$$

Since $E_B(\sigma) \equiv M \pmod{n}$, Alice has verified that the message is from Bob. This works because $E_B(\sigma) = (\sigma)^5 = (M^{29})^5 = M \pmod{91}$ by Euler's Theorem.